

GamutX

Code Quality

PYTHON SCRIPTING

Code Quality

- What is Code Quality?
- Benefits of Code Quality

What is Code Quality?

Code quality refers to **how well your code is written**—not just if it runs, but whether it's **clear, organized, and easy to improve later**.

Think of it this way:

- Writing code isn't just about getting it to work. It's also about making sure others (or future self) can:
 - Understand it quickly
 - Fix bugs without confusion
 - Add new features without breaking old ones

High-quality code helps:

- Keep the project **organized**.
- Make it **easier to fix bugs**.
- Make the codebase **scalable**, meaning it's easier to add new features over time.

Example

Before:

```
def do(x, y):  
    z = x + y  
    return z
```

→ X Unclear function and variable names

After:

```
def add_frame_counts(frame_start, frame_end):  
    """Adds start and end frame counts to get total frame range."""  
    total_frames = frame_start + frame_end  
    return total_frames
```

Clear, descriptive function name

easily clarify the inputs

Understands what this function does

Clear naming and a **docstring** make the purpose of the function obvious.

Benefits of Code Quality

Readability:

- Makes it **easier for others** (or **your future self**) to **follow** what the **code** is doing
- Useful in **collaborative environments** where **multiple people** work on the **same** tools

Maintainability:

- Reduces time spent **fixing** or **updating** **scripts**
- Easier to **extend** or **adjust** code when **project** needs **change**

Consistency:

- A common **coding style** across the team keeps everyone on the **same page**
- Reduces **confusion** and **learning curve** when **jumping** between projects

Bug Prevention:

- Clean code can help catch issues early—like **unclear variable** names or **risky logic**
- Reviewing quality also helps spot **hidden errors** before they **cause problems**

Key Aspects of Code Quality

- Readability
- Maintainability
- Consistency
- Documentation
- Reliability
- Efficiency

Readability

What it means:

code should be **easy** to **read** and **understand**—even for someone new to the project.

Clear, descriptive function name

✗ Bad Practice

```
--def a(x): return x*2
```

No docstrings

Unclear Names

✓ Good Practice

```
def double_value(number):  
    """Return double the given number."""  
    return number * 2
```

Best Practices:

- Use clear, meaningful names for variables and functions
- Follow naming conventions (e.g., snake_case in Python)
- Keep code cleanly formatted and properly indented

Why this matters:

- Readable code **reduces confusion**.
- It saves time because people don't need to guess what your code is doing.

Maintainability

What it means:

The **code should** be **easy to update** or **fix** without breaking other parts of the project.

Best Practices:

- Avoid hardcoding values
- Use functions to group logic
- Keep code modular and organized

✗ Bad Practice

```
tax = 0.05  
total = 100 + (100 * 0.05)
```

Hardcoded value

Logic not reusable

✓ Good Practice

```
TAX_RATE = 0.05  
  
def calculate_total(price):  
    return price + (price * TAX_RATE)
```

Constant

Reusable logic

Why this matters:

If something changes (like the **tax rate**), you **only update it once**—in one place.

Consistency

What it means:

- All code in the project **should follow the same style**.
- This includes naming, spacing, comments, and structure.

Follow a style guide like PEP 8 for Python.

✗ Bad Practice

```
def getUserData(userID):  
    user_data={"Name":"John","Age":30}  
    return user_data  
  
def get_user_data(user_id):  
    return {"name": "John", "age": 30}
```

inconsistent casing

inconsistent indentation

Duplicate Logic

✓ Good Practice

```
def get_user_data(user_id):  
    """Fetch user data by user ID."""  
    user_data = {  
        "name": "John",  
        "age": 30  
    }  
    return user_data
```

Use of snake_case

Clear docstring

Consistent format

Why this matters:

When everything **follows** the **same structure**, it's **easier** to **read, navigate**, and **collaborate** on.

Documentation

What it means:

Code has **clear documentation** explaining its **purpose**, **usage**, and **functionality**.

Best Practices:

- Add docstrings to functions and classes
- Include a README for bigger projects
- Document setup steps, dependencies, or known issues

Example:

```
def fetch_data(url):  
    """  
    Fetch data from the given URL.  
  
    Args:  
        url (str): The API endpoint URL.  
  
    Returns:  
        dict: Parsed JSON data.  
    """  
    pass
```

→ In this Docstring we clearly Describe

→ what **function** does

→ what **arguments** we passing

→ what we **returns**

Why this matters:

Anyone using your function can understand what it does, without reading every line inside it.

Reliability

What it means:

Your **code should** work **correctly** and **handle** problems without crashing.

Best Practices:

- Handle errors using try-except blocks.
- Write test cases for the main functionality.
- Validate inputs before using them.

✗ Bad Practice

```
value = int(input("Enter a number: "))
```

→ No error handling

✓ Good Practice

```
try:  
    value = int(input("Enter a number: "))  
except ValueError:  
    print("Invalid input. Please enter a valid number.")
```

→ using try except we handled bad inputs

Why this matters:

If a user makes a mistake, program should respond gracefully—not crash.

Efficiency

What it means:

Good code makes **smart use** of **memory** and **CPU**—especially important in VFX tasks.

Best Practices:

- Use the right data structures
- Avoid unnecessary loops or repeated work
- Optimize things like file reading or database queries.

✗ Bad Practice

```
squares = []  
for i in range(10):  
    squares.append(i**2)
```

→ Unnecessary loop

✓ Good Practice

```
squares = [i**2 for i in range(10)]
```

Use list comprehension ←

Why this matters:

Efficient code runs faster critical when dealing with large scenes or high-resolution assets.

Code Quality Is a Continuous Effort

- Code quality **isn't a one-time task**—it's something you build and maintain over time.
- Focus on **readability**, **maintainability**, **efficiency**, and **consistency**.
- High-quality code:
 - Is easier to **understand** and **update**
 - **Performs better**
 - Helps your team work **faster** and with **fewer bugs**

| Good code doesn't just work—it lasts.



Summary

Operator	Focus Area	Real-World Benefit
• Readability	→ Clear code structure and Understanding	→ Easier handoff and debugging
• Maintainability	→ Easy to update and scale	→ Less risk when making changes
• Consistency	→ Uniform style across the project	→ Better team collaboration
• Documentation	→ In-code explanations and project notes	→ Easier reuse and onboarding
• Reliability	→ Handles errors and edge cases	→ More stable tools
• Efficiency	→ Optimal performance	→ Faster tools for large-scale work

How to Improve Code Quality

- Coding Standards
- Refactoring
- Code Reviews

How to Improve Code Quality

- Improving code quality means making your code more **reliable**, **easier** to **maintain**, and **ready for growth**.

To Do It:

- **Follow Coding Standards** → Adhere to coding standards and best practices.
- **Refactoring** → Regularly refactor code to remove redundancy or improve structure.
- **Code Reviews** → Regularly review and get feedback from peers.
- **Automated Tools** → Use linters, formatters, and CI/CD pipelines.
- **Testing** → Write unit tests, integration tests, and regression tests.

Note: We'll cover **Automated Tools** and **Testing** in a later

Follow Coding Standards

What it means:

Coding standards are a **shared set of rules** that **define how code** should **look** and **behave**. They cover:

- Naming conventions (like snake_case)
- Indentation and spacing
- Commenting and documentation style
- Overall structure of your code

✗ Bad Practice

```
def calcArea(r):return 3.14*r*r
```

Bad name style

No spaces, no docstring

✓ Good Practice

```
def calculate_area(radius):  
    """Calculate and return the area of a circle."""  
    return 3.14 * radius * radius
```

Clear parameter

Proper spacing

Docstrings

Why It's Important:

- Helps keep the code consistent across teams
- Makes code easier to read and review
- Reduces back-and-forth during code reviews

Tools to Help:

- **flake8** – checks for style errors
 - **black** – formats code automatically
- 

Refactoring

What it means:

Refactoring means **restructuring code** without **changing** what it does, to make it **cleaner** and **easier** to work with.

Why It's Important:

- Eliminates repetition or messy code
- Makes your code easier to modify or extend
- Improves readability and performance

When to Refactor:

- Code is difficult to understand.
- Duplicate or repeated patterns emerge.
- Adding new functionality feels overly complex.

Before Refactoring

```
def calculate_sum(a, b):  
    return a + b  
  
def calculate_difference(a, b):  
    return a - b
```

- ✗ Code repeated
- ✗ Hard to extend later

After Refactoring

```
def calculate(a, b, operation):  
    if operation == "sum":  
        return a + b  
    elif operation == "difference":  
        return a - b
```

- ✓ Less repetition
- ✓ Easier to extend
- ✓ Cleaner, more flexible

Code Reviews

What it means:

- **Code review** is the **process** where **another developer looks** at **your code before it's merged** into the **main project**.
- The goal is to **catch mistakes**, **follow standards**, and **improve quality**.

Why It's Important:

- **Detects bugs** and **vulnerabilities early**.
- Encourages **knowledge sharing** among team members.
- Ensures **consistency** in **style** and **structure**.

Best Practices:

- Keep reviews **focused and short**.
- Provide **constructive feedback**, not **criticism**.
- Review both **functionality** and **style**.

Example Feedback:

| *"Consider renaming tmp to something more descriptive, like temporary_file."*

Tools Commonly Used:

- Pull Requests
 - GitHub
 - Bitbucket

→ These tools show code changes and allow inline comments

GitHub PR

Add more commits by pushing to the `master` branch on `Foxite/FluentEmail`.



✓ All checks have passed
1 successful check [Show all checks](#)

✓ This branch has no conflicts with the base branch
Merging can be performed automatically.

[Merge pull request](#) You can also [open this in GitHub Desktop](#) or view [command line instructions](#).

Bitbucket

Merge pull request

Commit message

Pull request #1: BLUES-273 - Handle errors when fetching dashboard details

Merge in BLUES/admin-dashboard from feature/BLUES-273-add-error-handling to master



☒ Delete source branch `feature/BLUES-273-add-error-handling` after merging

[Merge commit](#) [Merge](#) [Close](#)

MERGE STRATEGY

Merge commit (default)
--no-ff

Rebase and merge
rebase + merge --no-ff

Squash
--squash

Summary

Practice	Goal	Example Tool or Habit
• Coding Standards	→ Uses multiple space characters	→ flake8, black, follow PEP 8
• Refactoring	→ Clean up or simplify code	→ Remove repeated logic
• Code Reviews	→ Improve quality and share knowledge	→ GitHub pull requests, clear feedback

Code Quality - GIT

- Writing Descriptive Commit Messages
 - Guidelines for Writing Good Commit Messages

Write Descriptive Commit Messages

What's a Commit Message?

- A **commit message** is a short note we write when saving changes to our code **using Git**.
- It helps us and our team understand **what changed** and **why it changed**.

Why It's Important

- Helps track the history of changes
- Makes it easier to debug later
- Improves team communication
- Useful during code reviews

If commit **messages are vague, nobody**—including you—will understand the **change later**.

Guidelines for Writing Good Commit Messages

Be Clear and Specific

- Say **what we did** and **where**
- **Avoid** general terms like "fix" or "change" without context

✗ Bad Practice

```
git commit -m "Update"
```

This tells us nothing—

- what was updated?
- Why?
- Where?

✓ Good Practice

```
git commit -m "Fix bug in user authentication by adding validation for email format"
```

What was fixed

How it was fixed

Explain Why (When It Helps)

- If the reason for a change isn't obvious, include it briefly

✗ Bad Practice

```
git commit -m "Fixed stuff"
```

"**Stuff**" is not helpful. We won't remember (or understand) what it means later.

✓ Good

```
git commit -m "Refactor user authentication module to reduce duplication and improve maintainability"
```

What was Refactor?

Why it Refactor?

Provide Context in Commit Message

- Write the message like a command or instruction
 -  **“Add validation for email format”**
 -  **“Added validation”**

Bad Practice

```
git commit -m "Added validation"
```

Too vague — lacks detail

Good

```
git commit -m "Add validation for email format"
```

What the change Does

Where it is Added

Quick Tip

- If your commit message doesn't help someone else understand the change without reading the code—**rewrite it.**

Code Quality - Basic

- Optimize Import Statements
- Use Understandable Variable Names
- Use Consistent Naming Conventions
- Add Detailed Inline and Multi-line Comments
- Use Proper Spacing and Formatting
- Add Informative Print Statements
- Ensure Code is Self-Explanatory
- Use Constants

Optimize Import Statements

What It Means:

Only **import** what you **need**. This makes your **code cleaner** and can **reduce memory** usage.

✗ Bad Practice

```
from math import *
```

- This brings in **everything** from the **math** module
- even if we only need **one or two things**.

✓ Good Practice

```
from math import sqrt, pi
```

- ✓ Specific imports = better clarity + less risk.

Use Understandable Variable Names

What It Means:

- Variable names should describe their **purpose**.
- This improves **clarity** and **reduces** confusion.

✗ Bad Practice

```
a = 5  
b = a * 2  
c = "Hello"
```

unclear names

✓ Good Practice

```
number_of_items = 5  
double_items = number_of_items * 2  
greeting_message = "Hello"
```

Clear purpose

Easy to understand logic

Descriptive names help others (and us) understand the logic without needing extra explanation.

Use Consistent Naming Conventions

What It Means:

Ensure that naming conventions are **consistent throughout the codebase**.

✗ Bad Practice

```
def calculate_area(r):  
    pass  
  
def CalculatePerimeter(side):  
    pass
```

Mixing styles

- snake_case
- PascalCase

✓ Good Practice

```
def calculate_area(radius):  
    pass  
  
def calculate_perimeter(side_length):  
    pass
```

- Both function names use **lowercase** with **underscores**
- **consistent** and **easy** to follow.

Add Detailed Inline and Multi-line Comments

Why Comments Matter

- **Comments help** us (and our **teammates**) understand the purpose and **logic behind our code**.
- Even when the code is technically correct, **without comments**, it can be **hard** to follow—especially **months later**.

There are two main types:

- **Inline comments** – explain a **specific line or logic step**
- **Multi-line comments** – give a **bigger-picture explanation** of a code block

Inline Comment Examples

- Inline comments **clarify** specific parts of the **code**.
- When **used effectively** they **improve readability**,
- making **code easier** to follow **without requiring deep analysis**.

✗ Bad Practice

```
numbers = [1, 2, 3, 4, 5]
result = []
for num in numbers:
    if num % 2 == 0:
        result.append(num) # even
```



Comment here is too **short to be helpful**.

✓ Good Practice

```
numbers = [1, 2, 3, 4, 5]
result = []
# Loop through each number in the list
for num in numbers:
    # Check if the number is even
    if num % 2 == 0:
        result.append(num) # Add even numbers to the result list
```

- Now each step is **clear and purposeful**.
- Someone else reading the code won't need to guess.

Multi-line Comment Examples using -

- When we're working with a **block of logic** that **spans several lines**
- or includes **steps** that **build on each other**
- it's **better** to **use** a **multi-line comment**.

Multi-line Comment Examples using - `"""` **Triple Quotes**

- Adding **descriptive block** comments
 - Writing docstrings for **functions/classes**
 - Explaining **complex logic** or **steps**
- 

Multi-line Comment Examples using -

✗ Bad Practice

```
# This code handles stuff
# It does some things to input
# Adds results to a dict

user_input = "  john.doe@example.com  "
name_input = "john doe"

email = user_input.strip()
name = name_input.title()
user_data = {
    "email": email,
    "name": name
}
```

- ✗ Too vague
- ✗ Doesn't explain what the code does
- ✗ Generic wording like "stuff" or "some things"
- ✗ No mention of purpose or input/output

✓ Good Practice

```
# This block processes user input for a registration form:
# - It trims whitespace from the input
# - Checks for a valid email format
# - Converts the name to title case for consistency
# - Adds the cleaned data to a dictionary

user_input = "  john.doe@example.com  "
name_input = "john doe"

email = user_input.strip()
name = name_input.title()
user_data = {
    "email": email,
    "name": name
}
```

explain complex logic step-by-step —
perfect for teams.

Multi-line Comment Examples using - `''' '''` Triple Quotes

✗ Bad Practice

```
''' Script stuff '''  
data = [10, 20, 30]  
  
total = sum(data)  
print("Done")
```

*No actual detail about what
this script does*

✓ Good Practice

```
'''  
Calculate total score from a list of values.  
Useful for summarizing render stats or shot quality metrics.  
'''  
  
data = [10, 20, 30]  
  
total = sum(data)  
print(f"Total score: {total}")
```

- ✓ Clear purpose
- ✓ Describes what the code does
- ✓ Explains where it's used (render stats)

How to Write Effective Comments

Guideline

- **Explain decisions**
- **Avoid obvious comments**
- **Be precise**

What It Means

- **Focus on the why** — why a condition or method was used
- **Don't state things** the **code already makes clear** like `x = x + 1 # Increment x.`
- Use **short, targeted** explanations

Key Takeaways:

- ✓ Use **inline comments** to clarify tricky lines of logic
- ✓ Use **multi-line comments** to explain full blocks or sections
- ✗ **Avoid cluttering** the code with comments that state the obvious

Good **comments guide the reader** without repeating the code

Use Proper Spacing and Formatting

Importance:

- Consistent **spacing** and formatting **improve readability** and follow community standards (like **PEP 8 for Python**).

● Bad Practice:



✓ Good Practice:



Key Points:

- Follow consistent indentation and spacing rules.
- Place each statement on a new line for clarity.

Add Informative Print Statements

Importance:

- **Descriptive outputs** help **understand** the **results of the code**, especially for **debugging** and **logging**.
- This is also known as - Using **Detailed** and **Context-rich String Outputs**

● Bad Practice:

```
print("Error")
```

Too vague

No context or help

✓ Good Practice:

```
print("Error: Invalid user input. Please enter a valid email address.")
```

Clear Message

Tells what went wrong

Suggests how to fix it

Ensure Code Is Self-Explanatory

What It Means:

Code should be **clear enough** that it **doesn't need extra comments** to **explain simple operations**.

● Bad Practice:

```
# This function adds two numbers  
def add(a, b):  
    return a + b
```

Unnecessary comment

Generic names

✓ Good Practice:

```
def add_numbers(first_number, second_number):  
    return first_number + second_number
```

Clear variable names

Good naming reduces the need for obvious comments.

Use Constants

Why It Matters:

- Use **named constants** instead of **hardcoding numbers**.
- This makes the code **more readable** and **easier to maintain**.

● Bad Practice:

```
total_price = price * 1.2
```

What is 1.2?

- Hard to **understand** and **maintain**

✓ Good Practice:

```
TAX_RATE = 0.2  
total_price = price * (1 + TAX_RATE)
```

Constants

- Clear what the value means
- Easy to **update** later

Practice

- **Optimize Imports**
- **Use Clear Variable Names**
- **Consistent Naming**
- **Add Comments Where Needed**
- **Proper Spacing & Formatting**
- **Informative Print Statements**
- **Self-Explanatory Code**

Purpose

- Reduce memory and improve clarity
- Make code easier to read and understand
- Improve consistency across the codebase
- Explain non-obvious logic and decisions
- Follow community style and improve readability
- Give context during debugging or logging
- Reduce reliance on comments

Code Quality - Refactoring

- Remove Commented-out Code Before Pull Requests
- Remove Hardcoded Values

Remove Commented-out Code Before Pull Requests

Why It's Important

Leaving old, commented-out code in your file creates unnecessary noise. It:

- Makes the code harder to read
- Confuses other developers ("Is this still needed?")
- Clutters the codebase

● Bad Practice:

```
# def old_function(x):  
#     return x + 1  
  
def new_function(x):  
    return x * 2
```



This raises questions:

- Is the old function still needed?
- Should we bring it back?

✓ Good Practice:

```
def new_function(x):  
    return x * 2
```

- ✓ Clean and focused
- ✓ Use Git for old code

Remove Hardcoded Values

Why It's Important

Hardcoded values reduce flexibility and can introduce risks:

- They make our **code difficult to reuse** in other environments
- They may **expose sensitive data** (like API keys or passwords)
- They **make the code harder to configure or update**

● Bad Practice:

```
DATABASE_URL = "http://localhost:5432/mydb"
```

Hardcoded URL

- Not flexible or secure

✓ Good Practice:

```
import os
DATABASE_URL = os.getenv('DATABASE_URL', 'http://localhost:5432/mydb')
```

Uses env variable

Has a safe default

Code Quality - Logic Writing

- Use Context Managers for File Operations
- Add Docstrings
- Handle Edge Cases in Logic
- Avoid Repeating Loops
- Avoid Deep if-else Nesting
- Use List Comprehensions

Use Context Managers for File Operations

Why It's Important

- Automatically handles **opening** and **closing files**
- Prevents **issues** like **leaving files open** by **accident**.
- Makes **code cleaner** and **easier to read**

● Bad Practice:

```
file = open('file.txt', 'r')  
try:  
    data = file.read()  
finally:  
    file.close()
```

- Easy to forget close()
- Risk of file left open

✓ Good Practice:

```
with open('file.txt', 'r') as file:  
    data = file.read()
```

→ Use with to manage files safely and cleanly.

Add Docstrings

Why It's Important

- Docstrings explain what a **function does**, what the **inputs are**, and **what it returns**
- Especially useful for **more complex functions** or ones that handle **special cases**

● Bad Practice:

```
def calculate_area(r):  
    return 3.14 * r * r # area
```

- ✗ No explanation
- ✗ Inline comment is unclear
- ✗ Not helpful for others

✓ Good Practice:

```
def calculate_area(radius):  
    """  
    Calculate the area of a circle given its radius.  
    Formula: Area =  $\pi$  * radius^2  
    """  
    return 3.14 * radius * radius
```

Clear docstring

Descriptive names

- Useful especially for **public APIs** or **shared utility functions**.

Handle Edge Cases in Logic

Why It's Important

- Ensures code is **robust** and **won't crash unexpectedly**
- Makes Code **reliable** under **different** conditions

● Bad Practice:

```
def divide(a, b):  
    return a / b
```

- ✗ No error handling
- ✗ Unsafe if b is zero

✓ Good Practice:

```
def divide(a, b):  
    """  
    Divide two numbers.  
  
    Args:  
        a (float): Numerator  
        b (float): Denominator. Must not be zero.  
  
    Returns:  
        float: Division result  
  
    Raises:  
        ValueError: If b is zero.  
    """  
    if b == 0:  
        raise ValueError("Denominator must not be zero")  
    return a / b
```

→ Clear Docstrings

→ Handles edge case (b = 0)

Avoid Repeating Loops

Why It's Important

- **Unnecessary** repetition can **slow down** the code
- Cleaner logic improves **performance** and **clarity**

● Bad Practice:

```
items = ["A", "B", "C", "D"]

for i in range(len(items)):
    for j in range(len(items)):
        if i < j:
            print(items[i], items[j])
```

- Repeats unnecessary comparisons
- **Slower** and harder to follow

✓ Good Practice:

```
items = ["A", "B", "C", "D"]

for i, item_i in enumerate(items):
    for item_j in items[i+1:]:
        print(item_i, item_j)
```

Uses index + value

No need for != check

- This prints **each unique pair only once**,

Avoid Deep if-else Nesting

Why It's Important

- Deep nesting is **hard to read** and **maintain**
- **Flattening logic** makes the main **idea clearer**

● Bad Practice:

```
if a: → First layer
    if b: → Second layer
        if c: → Third layer
            do_something()
```

- ✗ Too many levels
- ✗ Hard to read

✓ Good Practice:

```
if not a:
    return
if not b:
    return
if not c:
    return
do_something()
```

Flat and readable logic

main action is now clear

- Use early returns to flatten logic.

Use List Comprehensions

Why It's Important

- Makes our **code shorter** and **easier to read**
- **Clearly expresses** intent when **creating a new list**
- **Ideal** for **simple loops** that **transform** or **filter data**

● Bad Practice:

```
squares = []  
for x in range(10):  
    squares.append(x * x)
```

✗ Too long for a simple task

✓ Good Practice:

```
squares = [x * x for x in range(10)]
```

→ List Comprehensions

- ✓ One line
- ✓ Easy to read
- ✓ Fast to write

Practice

- **Use Context managers for files**
- **Add docstrings**
- **Handle edge cases**
- **Avoid repeated loops**
- **Avoid deep nesting**
- **Use list comprehensions**

Why It's Valuable

- Handles files safely and cleanly
- Explain function purpose and usage
- Prevents crashes and unexpected errors
- Improves efficiency
- Simplifies logic flow
- Cleaner and more readable list creation

Code Quality - Functions

- Use Descriptive Names for Functions and Methods
- Use Functions to Encapsulate Logic
- Encourage Code Reuse (Default Libs)
- Ensure Code Modularity
- Use Meaningful Default Arguments
- Exception Handling within a Function
- Avoid Code Duplication
- Limit the Scope of Variables

Use Descriptive Names for Functions and Methods

Why It Matters:

- Clear function names make the code **self-explanatory**
- Reduces the need for **extra comments**

● Bad Practice:

```
def func(x):  
    return x * x
```

The diagram illustrates a bad practice. A code block shows a function named 'func' that takes a parameter 'x' and returns 'x * x'. An orange bracket on the left side of the code block is connected by a dashed orange arrow to an orange box containing the text 'Unclear purpose'. A green bracket is drawn under the expression 'x * x' in the return statement, with a dashed green arrow pointing down to a green box containing the text 'unclear names'.

✓ Good Practice:

```
def calculate_square(number):  
    return number * number
```

- We know **what this function does** just by reading its name.

Use Functions to Encapsulate Logic

Why It Matters:

- Breaks big **tasks** into **smaller, manageable** parts
- Makes **code easier** to **read, test, and maintain**

● Bad Practice:

```
def process_data(data):  
    cleaned_data = []  
    for item in data:  
        if valid(item):  
            cleaned_data.append(clean(item))  
    save_data(cleaned_data)
```

- Mixed logic
- Hard to read
- Not reusable

✓ Good Practice:

```
def process_data(data):  
    cleaned_data = clean_data(data)  
    save_data(cleaned_data)  
  
def clean_data(data):  
    return [clean(item) for item in data if valid(item)]
```

- Each function has a single, clear responsibility.

Encourage Code Reuse (Default Libs)

Why It Matters:

- **Saves time**
- Reduces **errors** by relying on **tested libraries**

● Bad Practice:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

→ Instead rewriting we can use
built in math module

✓ Good Practice:

```
import math  
  
def factorial(n):  
    return math.factorial(n)
```

↑ Built - in Module

→ Use built-in or third-party functions when possible.

Ensure Code Modularity

Why It Matters:

- **Separates** tasks into **independent** parts
- Makes **testing** and **reuse** easier

● Bad Practice:

```
def process_data():  
    data = read_data()  
    cleaned_data = clean_data(data)  
    save_data(cleaned_data)
```

- ❌ All logic inside one function
- ❌ Hard to test or reuse
- ❌ Can't swap parts easily

✓ Good Practice:

```
def process_data():  
    data = read_data()  
    cleaned_data = clean_data(data)  
    save_data(cleaned_data)  
  
def read_data():  
    pass  
  
def clean_data(data):  
    pass  
  
def save_data(data):  
    pass
```

Each task = separate function

✓ Easy to test, reuse, replace

Use Meaningful Default Arguments

Why It Matters:

- Makes **functions easier** to use without always **specifying all arguments**
- Encourages **flexibility**

● Bad Practice:

```
def connect_to_server(host, port=80):  
    pass
```

No docstring

Default value (80) is unclear

✓ Good Practice:

```
def connect_to_server(host, port=8080):  
    """  
    Connect to a server with the given host and port.  
  
    Args:  
        host (str): The server host.  
        port (int, optional): The port to connect to. Defaults to 8080.  
    """  
    pass
```

Clear default (8080)

Clear defaults + docstring = more usable functions.

Exception Handling within a Function

Why It Matters:

- Easier to understand
- Easier to test
- Easier to debug

● Bad Practice:

```
def process_order(order):  
    validate_order(order)  
    calculate_total(order)  
    process_payment(order)  
    send_confirmation_email(order)
```

→ ❌ Hard to test or debug

✅ Good Practice:

```
def process_order(order):  
    if not validate_order(order):  
        return False  
    total = calculate_total(order)  
    if not process_payment(order, total):  
        return False  
    send_confirmation_email(order)  
    return True
```

- ✅ Each step is clear
- ✅ Easy to follow
- ✅ Fail early if needed

Avoid Code Duplication

Why It Matters:

- Don't repeat code that **only changes slightly**
- **Similar logic** should be written **once and reused**
- Makes updates **easier** and **reduces** chances of **mistakes**

● Bad Practice:

```
def calculate_circle_area(radius):  
    return 3.14 * radius * radius  
  
def calculate_square_area(side):  
    return side * side
```

- Duplicates logic in two places

✓ Good Practice:

```
def calculate_area(shape, dimension):  
    if shape == 'circle':  
        return 3.14 * dimension * dimension  
    elif shape == 'square':  
        return dimension * dimension
```

- ✓ Combines related logic into one reusable function
- ✓ Easier to update or extend
- ✓ Cleaner and simpler

Limit the Scope of Variables

Why It Matters:

- Keeps **variables local** to where they're **needed**
- Reduces **risk** of **side effects** or **accidental** overwrites

● Bad Practice:

```
result = 0

def calculate_sum(numbers):
    global result --> Global Scope
    for number in numbers:
        result += number
    return result
```

- Can cause **side effects** if **reused** elsewhere
- Harder to **test** or **debug**

✓ Good Practice:

```
def calculate_sum(numbers):
    result = 0 --> local variable
    for number in numbers:
        result += number
    return result
```

Local variables make the function **safer** and **easier to test**.

Principle

- **Use descriptive names**
- **Break down logic**
- **Reuse existing functions/libraries**
- **Keep code modular**
- **Use meaningful defaults**
- **Keep functions focused**
- **Avoid duplication**

Benefit

- Easier to understand at a glance
- Improves structure and reusability
- Saves time, avoids bugs
- Makes testing and updates easier
- Simplifies function calls
- Easier to debug and maintain
- Reduces effort and inconsistency

Code Quality - Handling Exceptions

- Handle Exceptions Properly
- Handle Multiple Exceptions

Handle Exceptions Properly

Why It Matters:

- **Ensure exceptions** are handled appropriately to prevent the **application from crashing** and to **provide meaningful error messages**.

● Bad Practice:

```
def divide(a, b):  
    return a / b
```

- ❌ Crashes if $b = 0$
- ❌ No error message
- ❌ Unclear to the user what went wrong

✓ Good Practice:

```
def divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError:  
        return "Error: Division by zero is not allowed"
```

Prevents crash

Returns helpful message

Handle Multiple Exceptions

Why It Matters:

- Real-world code can **fail in multiple ways**
- We should handle **each error type** with an **appropriate message**

● Bad Practice:

```
def read_file(file_path):  
    with open(file_path, 'r') as file:  
        return file.read()
```

✗ Crashes if file doesn't exist

✗ No info on what went wrong

✓ Good Practice:

```
def read_file(file_path):  
    try:  
        with open(file_path, 'r') as file:  
            return file.read()  
    except FileNotFoundError:  
        return "Error: File not found."  
    except IOError:  
        return "Error: An I/O error occurred."
```

- **Clear messages** for each case
- User **knows what happened**

Summary

Practice

- **Handle specific exceptions**
- **Catch multiple error types**

Why It Matters

- Avoids crashes and gives helpful feedback
 - Handles various issues separately and clearly
- 

Code Quality – Documentation

- Document Edge Cases
- Document Arguments Clearly

Document Edge Cases

Why It Matters:

- Makes logic **clear** when **unusual** input or **behavior** occurs
- Helps **avoid crashes** or **incorrect outputs** in **automated tools**

● Bad Practice:

```
def get_frame_duration(total_frames, total_seconds):  
    return total_seconds / total_frames
```

✗ No explanation of what inputs are valid

✓ Good Practice:

```
def get_frame_duration(total_frames, total_seconds):  
    """  
    Calculate the duration of each frame based on total time and frame count.  
  
    Args:  
        total_frames (int): Total number of frames. Must be greater than zero.  
        total_seconds (float): Total playback time in seconds.  
  
    Returns:  
        float: Duration of each frame in seconds.  
  
    Raises:  
        ValueError: If total_frames is zero or negative.  
    """  
    if total_frames <= 0:  
        raise ValueError("Frame count must be greater than zero")  
    return total_seconds / total_frames
```

Clearly Documented Edge Case

Handles edge case with a useful error

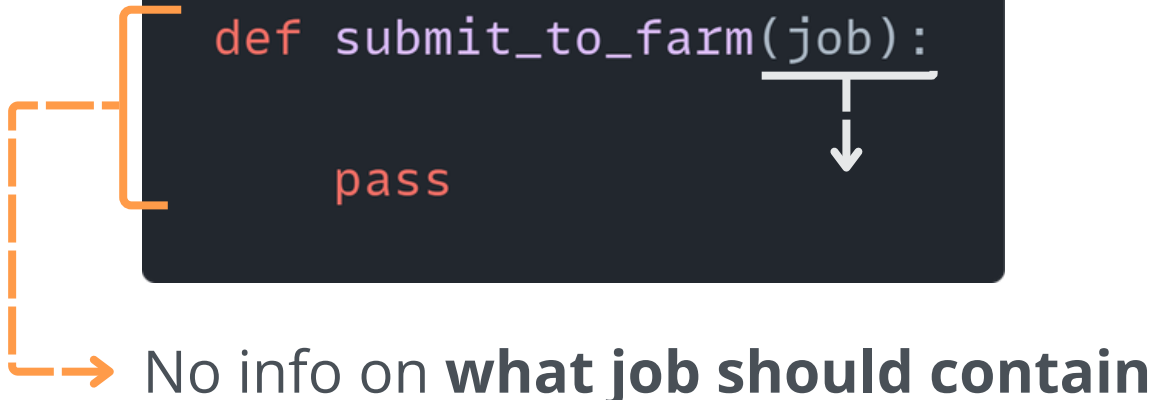
Document Arguments Clearly

Why It Matters:

- Shows what each **parameter** needs. Clarifies expected **types** and **structure**
- Prevents confusion and **misuse** of the function

● Bad Practice:

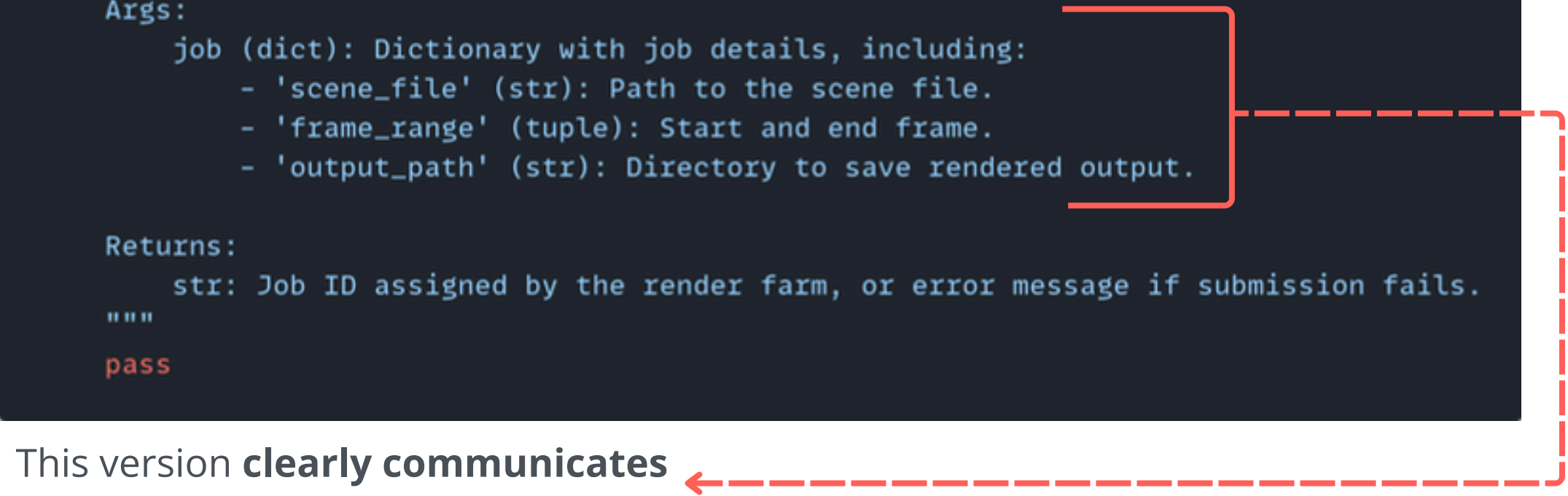
```
def submit_to_farm(job):  
      
    pass
```



No info on what job should contain

✓ Good Practice:

```
def submit_to_farm(job):  
    """  
    Submit a render job to the farm.  
  
    Args:  
        job (dict): Dictionary with job details, including:  
        - 'scene_file' (str): Path to the scene file.  
        - 'frame_range' (tuple): Start and end frame.  
        - 'output_path' (str): Directory to save rendered output.  
  
    Returns:  
        str: Job ID assigned by the render farm, or error message if submission fails.  
    """  
    pass
```



This version clearly communicates

- what the **function does**,
- **what it expects**, and
- **what it returns**.

Practice

- **Document edge cases**
- **Write API docstrings**

Why It's Important

- Prevents crashes in production with bad data
 - Makes public functions easier to understand and use
- 

Thank You

